

SIMCOMP: Sistema Integral de Gestión de Comparendos de Tránsito basado en Microservicios y Big Data

1st Deyton Riascos
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
deyton.riascos@uao.edu.co
2nd Samuel Izquierdo
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
samuel.izquierdo_b@uao.edu.co

3rd Mauricio Taborda
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
mauricio.taborda@uao.edu.co

Resumen—El presente documento describe el diseño, arquitectura e implementación de SIMCOMP (Sistema de Comparendos de Tránsito), una plataforma integral distribuida basada en una arquitectura de microservicios. El sistema está diseñado para ofrecer alta disponibilidad, tolerancia a fallos y balanceo de carga mediante su despliegue en un clúster de Docker Swarm. La solución integra un frontend en React, un API Gateway con Nginx, un balanceador de carga con HAProxy, múltiples microservicios backend en Node.js, bases de datos independientes en PostgreSQL y un servicio de análisis de datos masivos (Big Data) desarrollado con Apache Spark y Flask. Se presentan además los resultados de pruebas de carga con JMeter para validar la resiliencia del sistema.

Palabras clave—Docker Swarm, Microservicios, PySpark, API Gateway, Balanceo de Carga, HAProxy, Nginx, Node.js, React.

I. INTRODUCCIÓN

La gestión de comparendos e infracciones de tránsito a nivel gubernamental requiere sistemas capaces de procesar grandes volúmenes de datos, atender peticiones recurrentes de usuarios y garantizar la disponibilidad de la información ante posibles fallos de infraestructura. Las arquitecturas monolíticas tradicionales suelen presentar cuellos de botella al escalar estos procesos.

Para resolver esta problemática, se desarrolló SIMCOMP, una plataforma orientada a microservicios que separa las responsabilidades lógicas del sistema (Autenticación, Personas, Automotores, Infracciones, Comparendos y Reportes) garantizando despliegues modulares e independientes. Adicionalmente, se incorporó un componente analítico basado en Apache Spark para procesar y resumir grandes volúmenes de datos (Big Data) asociados a históricos de infracciones.

El despliegue de esta solución se llevó a cabo utilizando tecnologías de contenedorización y orquestación (Docker Swarm) sobre un entorno de máquinas virtuales aprovisionadas automáticamente a través de Vagrant, simulando un entorno de infraestructura en la nube.

II. ARQUITECTURA DEL SISTEMA

El ecosistema de SIMCOMP está diseñado bajo el patrón de arquitectura de microservicios.

A. Topología del Clúster (Docker Swarm)

La infraestructura virtualizada consta de tres (3) nodos operando en red privada:

- **Manager (192.168.100.2):** Actúa como nodo líder del Swarm. Aloja el balanceador de carga de capa 4 (HAProxy), el API Gateway (Nginx) y la aplicación Frontend.
- **Worker 1 (192.168.100.3):** Aloja los contenedores de la lógica de negocio (Node.js) y el motor de procesamiento de datos masivos (PySpark).
- **Worker 2 (192.168.100.4):** Nodo dedicado a la capa de persistencia de datos, alojando los clústeres de bases de datos PostgreSQL.

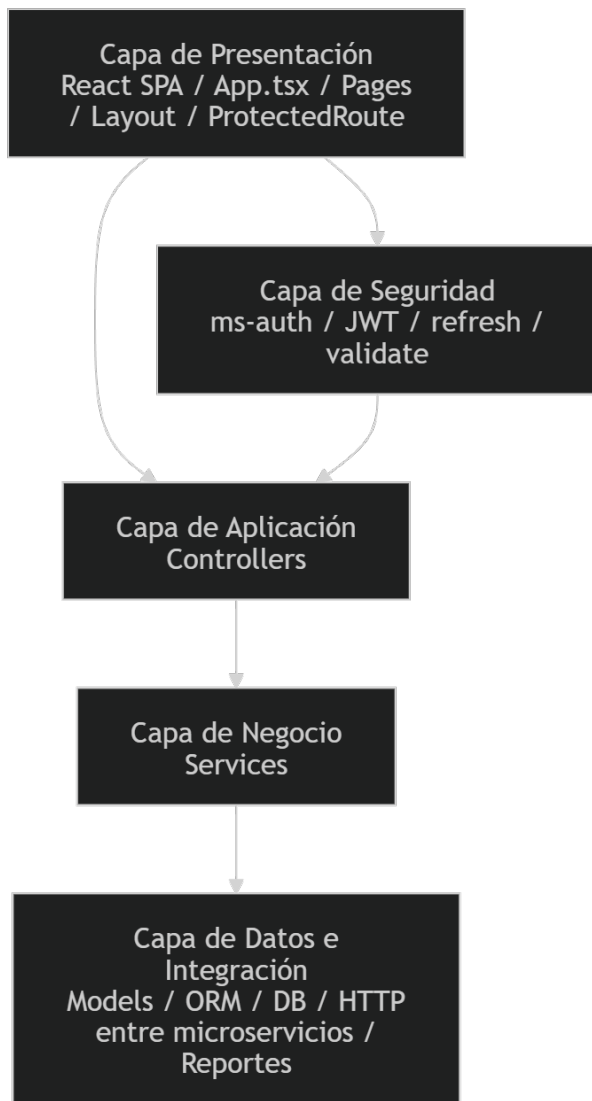


Figura 1. Diagrama de Arquitectura por Capas

B. Componentes Lógicos y Microservicios

El backend se encuentra dividido funcionalmente, y cada servicio posee su propia base de datos (Database-per-service) para mantener el bajo acoplamiento:

1. **Auth Service:** Manejo de usuarios y generación de JSON Web Tokens (JWT).
2. **Personas Service:** Gestión de ciudadanos y licencias de conducción.
3. **Automotores Service:** Registro de vehículos.
4. **Infracciones Service:** Catálogo de códigos de infracción.
5. **Comparendos Service:** Orquestador de la lógica de negocio y registro de multas, comunicándose con otros servicios de manera síncrona.
6. **Reportes Service:** Módulo encargado de consolidar reportes básicos.

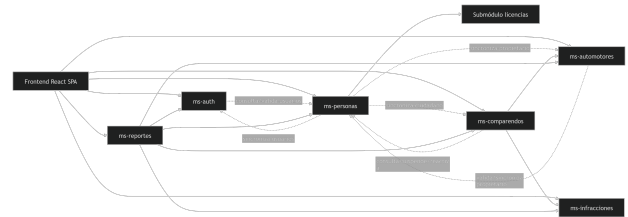


Figura 2. Relación y Comunicación entre Microservicios

C. Capa de Presentación y Balanceo

- **Frontend:** Aplicación Single Page Application (SPA) en ReactJS empaquetada con Vite.
- **HAProxy:** Actúa como punto único de entrada al clúster, balanceando la carga en round-robin hacia los distintos microservicios y exponiendo un panel de métricas.
- **Nginx (Gateway):** Proxy inverso encargado de interceptar las peticiones a la ruta `/api/*`, validar el token JWT y enrutar al microservicio correspondiente.

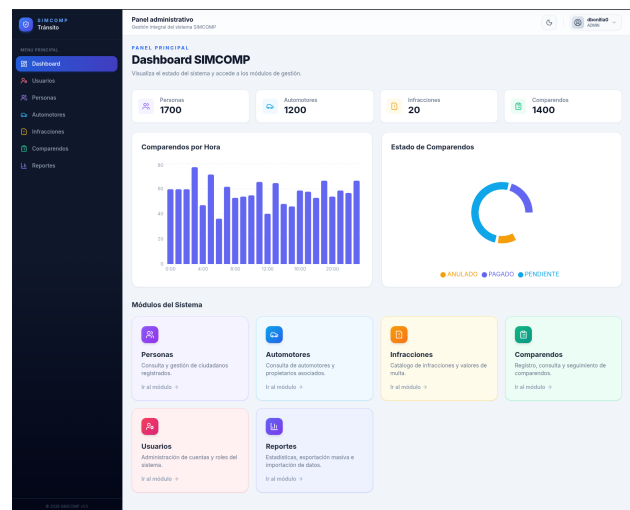


Figura 3. Interfaz de Usuario (Frontend React)

III. MÓDULO DE ANALYTICS Y BIG DATA (APACHE SPARK)

Para procesar el histórico de comparendos (dataset de gran volumen), el sistema incluye un servicio analítico aislado desarrollado en Python con Flask y PySpark.

A. Pipeline de Procesamiento

El pipeline analítico inicia con la ingesta de un archivo CSV alojado en un volumen de almacenamiento. PySpark se encarga de crear *DataFrames* distribuidos en memoria para realizar operaciones descriptivas como agregaciones, filtros y cruces.

El proceso sigue la siguiente ruta: Dataset CSV → API Flask → Apache Spark Engine → Consultas SQL en Memoria → Respuesta JSON → Renderizado (Chart.js)

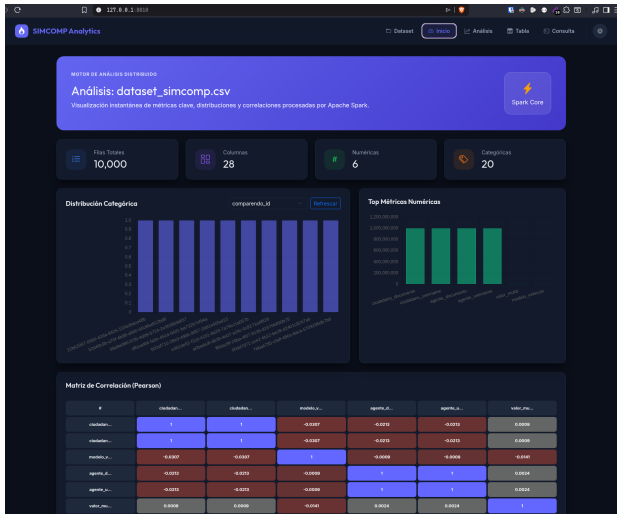


Figura 4. Dashboard de Analytics generado con PySpark

IV. DESPLIEGUE E INFRAESTRUCTURA COMO CÓDIGO

La orquestación general se automatizó usando **Vagrant** para crear la topología de red virtual, seguido de un aprovisionamiento (Provisioning) mediante scripts Bash y docker stack deploy.

La seguridad en el ecosistema se garantizó evitando el uso de archivos `.env` para datos críticos en producción, optando por el uso de **Docker Secrets** para inyectar contraseñas de bases de datos y claves privadas del cifrado JWT directamente a la memoria de los contenedores, aumentando la robustez contra posibles brechas de seguridad.

V. PRUEBAS DE CARGA Y RENDIMIENTO

Para evaluar el comportamiento del balanceador de carga y la tolerancia a fallos del sistema, se ejecutaron pruebas de estrés (Stress Testing) con Apache JMeter.

A. Resultados del Balanceo (HAProxy)

Se simularon peticiones recurrentes al endpoint de autenticación y de consulta de comparendos. A través del panel de métricas de HAProxy se observó la distribución equitativa de peticiones entre las réplicas del `ms-auth-service` y `ms-comparendos-service`.

VI. CONCLUSIONES

La adopción de una arquitectura de microservicios con orquestación en Docker Swarm demostró ser una solución altamente eficiente para el sistema SIMCOMP. Permitió escalar servicios críticos de forma independiente (replicación horizontal) sin afectar la disponibilidad del resto del ecosistema. La integración de Apache Spark evidenció la capacidad del sistema para no solo realizar procesamiento transaccional tradicional (OLTP), sino también procesamiento analítico (OLAP), entregando visualizaciones procesadas en tiempo récord. El uso conjunto de HAProxy, Nginx, Node.js y Docker Secrets conforman un stack sólido que garantiza rendimiento, seguridad y escalabilidad en escenarios de despliegue reales.

REFERENCIAS

- [1] M. Fowler y J. Lewis. (2014). *Microservices*. [Online]. Disponible en: <https://martinfowler.com/articles/microservices.html>
- [2] Docker Inc. (2023). *Docker Swarm mode overview*. [Online]. Disponible en: <https://docs.docker.com/engine/swarm/>
- [3] Apache Spark. (2023). *Spark Overview*. [Online]. Disponible en: <https://spark.apache.org/docs/latest/>
- [4] HAProxy Technologies. (2023). *HAProxy Configuration Manual*. [Online]. Disponible en: <http://cbonte.github.io/haproxy-dconv/>